

UNIVERSITY OF BONN

CAISA LAB

NATURAL LANGUAGE PROCESSING LAB

SUMMER SEMESTER 2026

FINAL REPORT

Trajectory-Level Failure Analysis of Scientific Agent Teams

GROUP MEMBERS:

Jiadong Han

3198936

Mohammad Saeed Motevali Amin

50246276

Instructor: Prof. Dr. Lucie Flek

Advisor: Dr. Shivam Rawat

September 23, 2026



1 Introduction

A failed run can mean three different things: the team could not solve the task, it could not operate its tools, or it lost an answer that had already passed a check. The final score does not separate them, yet each calls for a different remedy. A stronger model alone need not prevent misrouted messages or the loss of verified work. Only the trajectory tells them apart.

We study a Reasoner–Engineer–Critic team in mathematics and astrophysics. The Reasoner proposes a solution, the Engineer implements and tests it, and the Critic reviews the result. In mathematics, the team must produce a formal proof. In astrophysics, it must recover a planetary system from noisy observations. These tasks need different checks, but both require the agents to use feedback and pass useful work to the next role.

Recent mathematics projects also make this coordination problem visible. Anthropic’s Fermat formalisation report describes early teams losing project state and failing to collaborate [1]; OpenAI’s Navier–Stokes announcement combines a written argument with Lean formalisation [2]. These reports motivate studying the workflow. Our experiments do not evaluate their claimed mathematical results.

We therefore ask which outcome a failing score hides, and how that changes as the backbone model gets stronger. Every count comes from the recorded actions; correctness is checked after the agents stop: a Lean kernel replay of the proved statement and its axioms, and an offline rescoring of the fits the team already had. In these runs, workflow collapse fades as models get stronger, review failures remain a workflow bottleneck, and work that has already passed a check can be lost at every level.

2 Related Work

Agent failure analysis. Raj et al.’s *Model or Harness?* locates a failure by its interaction, fault side and mode [3]. MAST groups failures into system design, coordination and verification issues [4]. We use these labels to explain concrete events. Our six trace patterns organise the findings across a capability ladder; they are not a new root-cause taxonomy. AgentRx, *Which Agent Causes Task Failures and When?*, and TRAIL study failure attribution and its evaluation [5–7]. Their results do not validate our labels in scientific workflows. Our counts come from recorded events alone; model-assisted review and log inspection explain the counted behaviour without generating counts.

Review and correction. Intrinsic self-correction can damage correct answers in some settings [8]. Tool-supported criticism can improve correction, but can also introduce wrong revisions [9]. Our question is how a separate Critic uses the Engineer’s evidence: whether it preserves a successful proof check, asks for a useful revision, or makes a diagnosis that the feedback does not support.

Scientific checks. FATE and LeanCat provide formal mathematics tasks [10, 11]. Lean checks a proof term against a formal proposition [12]; the intended statement and allowed axioms still need separate attention [13]. STARGAZER studies recovery of planetary systems from radial velocities [14]. *Plausible but Wrong* shows why apparently successful astrophysics workflows can still contain scientific errors [15]. These studies support checking what a team concludes from its tools, as well as whether its final answer passes.

3 Methodology

Record, count, verify. We record every message, tool call, handoff, checker result and stop. Patterns are counted from those actions rather than from a model’s opinion of them: each of the six patterns in Section 6 is a condition on the recorded trace. Correctness is then assessed through offline validation. Model reviewers and readers of representative and disputed logs interpret the counted behaviour; they do not generate counts. Repeated behaviour helps locate a problem, but does not by itself explain the cause.

One design, two domains, one ladder. The same three roles, the same free routing and the same submission rule are used in both domains; only the tools change. Each team is run with a weak, a middle and a frontier backbone, so a trace pattern can be read against capability rather than against a single model. A further team is identical to the frontier team except for its Critic, which isolates the reviewing role from the rest. Figure 1 shows the roles, their tools and the ladder.

Produced and delivered work. In mathematics, a logged accepted check means that `check_lean` reports compilation without `sorry` for code declaring the task’s theorem name. We count whether Critic approval follows; this is our delivery measure. Because the harness check does not confirm that the proved statement is the intended one, we replay every accepted proof offline through the Lean kernel and require three further conditions: it compiles, it declares the task’s statement unchanged, and `#print axioms` reports no axiom beyond Lean’s three standard ones. This offline replay determines correctness. For the repair measurement, completion also requires normal termination.

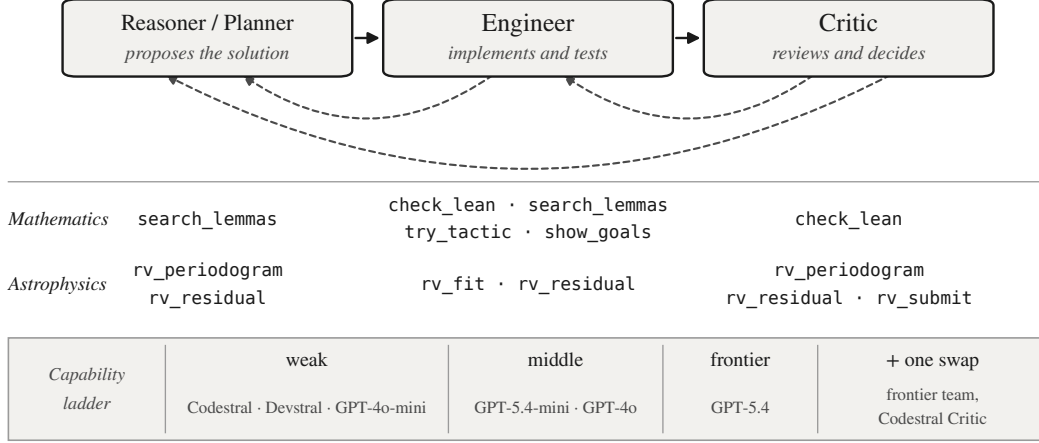


Figure 1: The team design. Feedback can return work to an earlier role. Only the Critic can submit, and its attempts are capped. The same design runs at three capability levels, plus one team whose Critic alone is swapped.

In astrophysics, we rescore submissions and subsets of planets already returned by the fitter. The offline check uses hidden truth to find whether any such subset passes all grading conditions. No new fit is produced. We count a loss when a passing subset exists but no actual submission passes. This shows a gap between the fitting outputs and the submitted answer; the agent may not know which subset is correct.

Naming a behaviour. Where a count needs a name, we borrow Raj et al.’s failure modes [3] and MAST’s broader System Design Issues category [4], describing the observed action before assigning one. Tool Feedback Neglect means ignoring available tool feedback; Reasoning Failure means a wrong inference; State Tracking Failure can include losing established task state or repeating actions without recognising the lack of progress. A label is an interpretation, not proof of a cause, and a role identifies where a problem appears, not whether the model, tool or grader is responsible. Appendix A describes the reviewed groups, retained-label counts and review process.

4 Data Set

Mathematics. MiniFATELeanCat contains 30 tasks from FATE-M, FATE-H and LeanCat [10, 11]. The Reasoner’s natural-language plan is a different object from the Engineer’s formal proof, and the compiler checks the proof, not the statement: kernel acceptance establishes the formal proposition under its dependencies, not that the proposition expresses the intended task [12].

Astrophysics. Our local bank has 100 synthetic radial-velocity systems derived from STARGAZER, separate from its published inventory. The observations give times, a star’s line-of-sight velocities and reported measurement errors. Planets make their star move; the team must recover the planet count and orbital parameters. The measured velocity combines orbital signals, instrument offsets and noise. Recovery depends on sampling, signal strength and how much of each orbit was observed [16]. Some outer planets complete less than one orbit during the observations.

The grader requires improvement over a constant-velocity baseline using the Bayesian information criterion (BIC), a low residual root mean square (RMS), the required planet count and a match to hidden orbital parameters. The RMS limit is 1.5 times the median reported measurement error [14, Section 3.3]. BIC penalises extra parameters. Neither a lower residual nor improvement over a constant-velocity baseline establishes recovery of every true planet. Short or noisy observations can support several fitted systems.

5 Experimental Setup

Both domains use a Reasoner–Engineer–Critic team. Agents choose when to hand work to another role and can return it for revision. Each team runs one model in all three roles at one of three capability levels: weak (Codestral, Devstral, GPT-4o-mini), middle (GPT-5.4-mini, GPT-4o) and frontier (GPT-5.4). This ladder is the analysis axis throughout Section 6, letting a trace pattern be read against capability rather than against one model; one mathematics configuration changes only the Critic. The levels are descriptive groupings; the experiments do not measure general intelligence.

Mathematics. Tools are granted per role. The Reasoner plans the solution and its formalisation and holds `search_lemmas`, a semantic search over library results. The Engineer holds all four: `check_lean` to compile a snippet, `try_tactic` to find a tactic that closes a stated goal, `show_goals` to inspect the proof state, and `search_lemmas`. The Critic holds `check_lean` alone, so it can re-verify the Engineer’s proof but cannot search or experiment; only its approval delivers a proof.

We run two mathematics experiments. The five-configuration comparison uses the same 20 tasks, with three attempts per task and 60 runs per configuration. The focused Critic comparison is the one controlled contrast in the paper: GPT-5.4 as Reasoner and Engineer in both teams, with either GPT-5.4 or Codestral as Critic, 154 runs each on the same 16 tasks. Table 1 combines both experiments: 214 runs for each of these two teams and 608 overall. Pairing uses experiment, statement and trial index. Repeated attempts are not independent theorems. Prompts, temperature 0.2 and the 30-turn cap are shared. The swap changes one role’s model, not expertise alone, and each run generates its own upstream work, so the two Critics do not receive identical saved inputs.

Reasoner	Engineer	Critic	Runs
GPT-5.4	GPT-5.4	GPT-5.4	214
GPT-5.4-mini	GPT-5.4-mini	GPT-5.4-mini	60
Codestral	Codestral	Codestral	60
Devstral	Devstral	Devstral	60
GPT-5.4	GPT-5.4	Codestral	214

Table 1: Mathematics: model assignments and total runs. The first and last rows differ only in the Critic.

Astrophysics. The Reasoner, named `planner` in this harness, proposes a planetary system and holds `rv_periodogram`, which returns candidate periods with their aliases, and `rv_residual`, which reports what a fitted model leaves behind. The Engineer holds both of those and `rv_fit`, which fits one planet per supplied period and is the only tool that produces orbital parameters. The Critic holds `rv_periodogram` and `rv_residual`, so it can re-run the diagnostics rather than trust the Engineer, and `rv_submit`, which no other role may call and whose attempts are capped. A periodogram peak is a starting hypothesis, not a complete orbit solution [17]. Tools are fixed during the evaluated runs: agents cannot write Python or change the fitter, which differs from STARGAZER’s original coding environment [14, Section 3.2].

Table 2 lists the four model configurations and their 229 runs. The later runs use filtered tasks and a lower match threshold, and the filter checks the match score, not every grading condition. Turn limits range from 30 to 90, and submission limits from 3 to 10. Task difficulty, filtering, thresholds and stopping rules differ, so the astrophysics configurations are not directly comparable: we report no ranking of models in this domain. Appendix B gives the task coverage, repetitions and caps.

Model in all three roles	Runs	Different tasks	Match threshold
GPT-5.4	145	49	0.80
GPT-5.4-mini	16	16	0.512
GPT-4o-mini	56	56	0.512
GPT-4o	12	12	0.512

Table 2: Astrophysics: model assignments, runs and grading thresholds. The first two rows are batches 1 and 2 discussed below. Four GPT-4o files contain only partial logs and are excluded.

We did not run a single-agent or matched no-Critic baseline in either domain.

6 Results

6.1 Overview of the observed patterns

Table 3 defines the six trace patterns as conditions on the recorded trace, and three findings organise what follows. Workflow collapse fades with capability: it dominates the weak teams and is absent from the frontier team’s protocol counters. Review failures do not fade but change character, from damaging rewrites to approvals claiming checks that never ran. Work that has already passed a check is lost at every capability level in both domains. Patterns can overlap, and two of them (approval without a new check, and repeated checking) are not errors by themselves: an earlier check may be sufficient, another may be useful.

Trace pattern	Condition on the recorded trace	§
The team cannot run itself	Invalid tool calls, failed handoffs or repeated messages prevent the next useful action.	6.2
A verified answer is lost	In mathematics, a successful check has no later approval. In astrophysics, a passing fitted subset exists but no submission passes.	6.3
The Critic approves without a new check	The Critic approves without its own recorded check. Earlier Engineer evidence must be considered.	6.4
The Critic revises accepted work	The Critic requests or makes a change after a successful check; a concrete rewrite can damage the result.	6.4
The Critic repeats checks without a new finding	The Critic repeats successful checks and reports no error in this cohort.	6.4
The verdict claims more than the checks	The verdict asserts properties, such as no added axioms, that the available tool outputs do not establish.	6.4

Table 3: The six trace patterns: observable conditions on the trace, not mutually exclusive root causes or a pooled model ranking. Patterns 1–2 are read across the capability ladder. Patterns 3–6 are quantified by the controlled Critic swap and corroborated in the uniform Codestral, Devstral, GPT-5.4-mini and GPT-5.4 teams.

6.2 Workflow and handoff failures

In the weaker mathematics teams, tool use and handoffs often fail before the agents can carry out the plan. Calling an unavailable tool fits *Tool Hallucination* [3]. In four Devstral runs, an Engineer self-handoff leaves an invalid message order and the provider rejects the next request. The review labels this routing mistake *Delegation Failure*, applying Raj et al.’s broader coordination category. The service error follows that mistake; it does not show that the service was unavailable.

The protocol fails in three further ways in the same cohort. In six Codestral runs an agent writes a tool call as prose inside a message instead of invoking it, so no tool runs. Six runs contain no tool call at all, and in three of those (all trials of one task) the Critic still returns an approval verdict over work that was never checked.

Another chain begins with ignored feedback: the checker explains why a tactic fails, but the Engineer repeats it. This behaviour receives the most common label in both weak teams (*Tool Feedback Neglect*; Appendix A). The run does not recover. Codestral ends 47/60 runs as **stuck** and three more as **framework_stop**, 83% in all. A stopping status alone does not say which failure occurred.

The stronger model, GPT-5.4, operates the workflow more reliably: all 214 mathematics runs record zero errors in the four saved protocol counters. The difficulty can then move to formalisation. The Reasoner has a useful plan, but the Engineer cannot turn a required step into library-level proof code. *Domain Knowledge Deficit* describes this gap [3] and is the most common reviewed label for both the frontier and middle teams (Appendix A). The Engineer’s repair attempts can then consume the remaining budget. Workflow collapse therefore fades with capability: the weak teams are blocked by the workflow, the stronger teams reach the task and are blocked by it. The division is not strict: a weak team’s first plan can already be wrong, before any tool is called.

The same ordering matters in astrophysics. Four of 16 GPT-5.4-mini runs never reach the Critic. In one case the Reasoner treats a residual check on an unfitted guess as evidence that the model failed to remove a signal (*Reasoning Failure*). The Engineer then obtains a useful fit but rejects it using that same residual (*Tool Feedback Neglect* [3]), a second error rather than a continuation of the first. Later messages repeat the diagnosis and no answer is submitted. In another case the Reasoner treats a generic grader hint as a physical diagnosis (*Reasoning Failure*); the team then abandons useful tests (*Under-initiative*). Appendix D identifies the cases and the diagnostic checks.

6.3 Producing an answer and missing it

In three matched trials of one mathematics task, the Engineer produces a proof that passes its check. The Codestral Critic sees a deprecation warning and rewrites the theorem. The replacement fails; later attempts do not recover the accepted proof and the run ends stuck. GPT-5.4 controls see the same warning and approve the unchanged proof. The break occurs at review: accepted work reaches the Critic, and the Critic changes it with no proof error to repair. The chain is unnecessary rewrite → rejected replacement → failed recovery, and in one trial the Critic resubmits its own rejected code five times after explicit type-error feedback. This explains a failed recovery; it does not make every missing approval a Critic error. Appendix A gives the matched trials and checker responses.

Astrophysics shows a related gap between fitting and submission. For GPT-4o-mini, the offline scorer finds a passing subset among existing fitting outputs in 40/56 runs. Ten of those 40 never submit a passing answer, a loss of 25%. The team may submit a fitted planet together with an extra component. The combined answer can fail the planet-count check even when a subset passes. Rechecking the same combined answer does not remove the extra component. This check uses hidden truth, so the existence of a passing subset does not show that the team could recognise it unaided (matching details in Appendix D). The ten losses are not automatically ten Critic mistakes. A more sharply localised loss occurs in the GPT-5.4-mini batch: after a useful four-planet fit, the Engineer omits the required handoff, so the answer never reaches the role that can submit it (*Instruction-Following Failure* [3]).

The same distinction matters for stronger teams. In mathematics, five of 161 runs with an accepted check receive no later approval, about 3%. All five end at the cap; two have no Critic action after the successful check. In the GPT-5.4 astrophysics batch, two of 102 runs with a passing subset never submit a passing answer, about 2%. These domain-specific checks measure different outcomes. They show why the final failure score alone cannot tell whether the team never found useful work or failed to deliver it.

The lost proofs were valid. In our Lean kernel replay (Section 3), all 24 mathematics runs that produced a task-matched, compiling, sorry-free proof and never obtained approval pass the full gate: they compile, they prove the task’s own statement, and `#print axioms` reports nothing beyond Lean’s three standard axioms. Those 24 span weak, middle and frontier teams, so this loss persists across capability levels.

6.4 What the Critic does with the evidence

Review can fail in two opposite directions: refusing correct work and certifying unchecked properties. The Critic swap examines the first; the kernel replay exposes the second.

Refusing correct work. Changing only the mathematics Critic leaves the team producing almost the same work: 161 accepted proof checks against 159. Yet the number of runs with an accepted proof but no approval rises from five to fifteen (Table 4). Approval is missing for about one accepted proof in ten with the weak Critic and one in thirty with the frontier Critic; approval rates fall from 97% to 91%. This is a descriptive comparison over repeated tasks, not a general effect estimate; Appendix A reports the matched-pair outcomes.

Math configuration	Accepted check	Later approval	No approval	Approval rate
GPT-5.4 in all roles	161	156	5	96.9%
GPT-5.4 + Codestral Critic	159	144	15	90.6%

Table 4: Each group has 214 runs across both experiments. Rates are approval after logged acceptance, not overall solve rates. The accepted runs differ between groups; the count does not verify final-proof identity or normal termination.

Across the weaker and mixed configurations, 25 runs contain a non-approval Critic message; 23 of those already held an accepted task-proof check, and all 23 pass the full kernel gate. Not one rejection in this cohort removed a bad proof. The three rewrite trials above show the mechanism: the Critic edits a proof it did not need to touch, the replacement fails, and the run never recovers.

Certifying what was not checked. The frontier Critic makes the opposite mistake. It issues 156 verdicts, all approvals, and rechecks in 153 of the 159 runs where it acts, finding no error in any of them; six approvals carry no recheck at all, which is not an error by itself when the earlier evidence is sufficient. Its verdicts nonetheless state that a proof “adds no axioms” or “matches the intended statement exactly”, and `check_lean` reports neither. The weak teams make the same claim with nothing behind it at all: three of the Codestral runs without a single tool call still end in approval. In the Critic comparison, 300 approved proofs are replayed; 13 (4.3%) fail the kernel gate: twelve assert compiled evaluations as named axioms through `native_decide`, and one is axiom-clean but proves a different statement. The twelve assert true arithmetic, and `decide` closes the same goals axiom-free, so what failed is the claim, not the mathematics.

Review failure therefore does not fade as the backbone improves; it changes character. The weak Critic rejects, and everything it rejected was valid. The frontier Critic never rejects but certifies properties its tools do not establish.

The astrophysics verdicts show the same gap between claim and check. The first batch runs against a fixed fitter the team cannot modify, which optimises agreement with the observations while the grader also requires agreement with the hidden planets; this is a condition of the tool setup, not of the team (Appendix D). A reference fit built from the true periods passes none of the 17 first-batch datasets, so part of this batch’s failure belongs to the fitting procedure rather than to the team. On seed20, and on three tasks in the filtered batch, the grader rejects even the reference answers: its error limit is 1.5 times the median reported uncertainty, and those uncertainties omit some of the injected noise. What the team adds is the inference from a rejection to a

physical cause. An unsupported “wrong period” claim appears in 38 of 45 reviewed failed runs: on seed12 the fitted period is 5.173 days against 5.164 injected, yet the Critic blames a half-period alias. The Critic states the unsupported explanation and the other roles act on it, producing new guesses and repeated fits.

Of the eleven failed runs in batch 2, only three are team mistakes of the kind described above. Three involve tasks whose reference answers fail the grader. Five require a weak or partly observed planet that the observations may not determine. We do not count those eight as model failures; they bound what this domain’s numbers can support (Appendix D).

6.5 How the team uses its loop and role boundaries

Where it worked. Among runs whose first returned Engineer proof check fails compilation, Codestral later completes 2/46 (4%), while GPT-5.4 completes 61/111 (55%). The stronger team can use the repair loop more often in these runs. This comparison is not a controlled test of adding the loop.

Return handoffs appear especially in tasks that need another hypothesis. Across the available astrophysics logs, 18/27 solved two-planet runs contain an explicit Engineer-to-Reasoner return, compared with 0/114 solved one-planet runs. This supports examining the role boundary during iterative work. It does not show that the return caused success. These counts pool the available logs across arms with different settings.

Where it did not. The GPT-5.4 mathematics Critic makes 153 of the team’s 644 proof-check calls, about 24%, with no rejected Critic check result. In the recorded runs, roughly one quarter of the team’s calls repeat successful verification. Whether they would catch errors in other runs, or whether removing the Critic would help, was not tested.

In astrophysics, none of the 229 retained runs exhausts its submission budget. Yet only the Critic can submit, and some runs stop before reaching it or before submitting a useful fitted subset. Unused attempts therefore belong in the workflow analysis, alongside the final score. They do not by themselves prove lack of initiative or show that another submission would help. The results locate where the team stops; they do not isolate the effect of the submission gate.

7 Conclusion

Score the handoffs, not only the answer. The final answer hides how the team reached it. We need to check whether the agents can operate their tools, preserve useful work across handoffs, and reach the role that can finish. In the weaker group, basic workflow failures often appear first. Stronger models more reliably reach the task, where reasoning and domain knowledge become clearer limits. The Critic can still block delivery after useful work has been produced.

The Critic is the coordination role. It approves work or asks for revision. In astrophysics, only the Critic can submit. A reviewer that misreads a warning or gives an unsupported physical diagnosis can make the result worse. Domain expertise is a plausible explanation, but our Critic swap changes more than expertise alone. The findings support preserving accepted work and requiring evidence for revision requests.

Assign roles around iterative work. The repair and return-handoff observations motivate specific roles for proposing another hypothesis or revising an implementation using feedback.

Add a role when the task needs a second hypothesis, not for safety. An extra reviewer is not automatically a safeguard: the frontier Critic repeats a quarter of the team’s successful checks without finding an error, and the weak Critic’s only rejections were of work that was already valid.

Limits. We ran no single-agent or no-Critic baseline of our own (Section 5). The mathematics experiments repeat 20 statements, so trials are not independent theorems and the effective sample is small. The astrophysics arms differ in tasks, thresholds and budgets, preventing direct comparisons. Their data are synthetic: known generating parameters give an answer key, not a demonstrated blind recovery method. Reviewer agreement measures a labelling process, not annotation accuracy, and historical software versions are not fully recorded.

References

- [1] Anthropic. Formalizing Fermat’s last theorem. <https://www.anthropic.com/research/formalizing-fermats-last-theorem>, September 2026. Research announcement, September 4; accessed September 13, 2026.
- [2] OpenAI. On the Navier–Stokes millennium prize problem. <https://openai.com/index/navier-stokes-solution/>, September 2026. Research announcement, September 8, updated September 10; accessed September 13, 2026.
- [3] Harsh Raj, Vipul Gupta, Anas Mahmoud, Razvan-Gabriel Dumitru, Darvin Yi, Aakash Sabharwal, and Yunzhong He. Model or harness? an interaction-centric taxonomy for localizing agent failures. arXiv:2607.28802v1, 2026.
- [4] Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. Why do multi-agent LLM systems fail? arXiv:2503.13657v3, 2025.
- [5] Shraddha Barke, Arnav Goyal, Alind Khare, Avaljot Singh, Suman Nath, and Chetan Bansal. AgentRx: Diagnosing AI agent failures from execution trajectories. arXiv:2602.02475, 2026.
- [6] Shaokun Zhang, Ming Yin, Jieyu Zhang, Jiale Liu, Zhiguang Han, Jingyang Zhang, Beibin Li, Chi Wang, Huazheng Wang, Yiran Chen, and Qingyun Wu. Which agent causes task failures and when? on automated failure attribution of LLM multi-agent systems. arXiv:2505.00212, 2025.
- [7] Darshan Deshpande, Varun Gangal, Hersh Mehta, Jitin Krishnan, Anand Kannappan, and Rebecca Qian. TRAIL: Trace reasoning and agentic issue localization. arXiv:2505.08638, 2025.
- [8] Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet. In *International Conference on Learning Representations*, 2024.
- [9] Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Nan Duan, and Weizhu Chen. CRITIC: Large language models can self-correct with tool-interactive critiquing. In *International Conference on Learning Representations*, 2024.
- [10] Jiedong Jiang, Wanyi He, Yuefeng Wang, Guoxiong Gao, Yongle Hu, Jingting Wang, Nailin Guan, Peihao Wu, Chunbo Dai, Liang Xiao, and Bin Dong. FATE: A formal benchmark series for frontier algebra of multiple difficulty levels. arXiv:2511.02872, version 4, 2026.
- [11] Rongge Xu, Hui Dai, Yiming Fu, Jiedong Jiang, Tianjiao Nie, Junkai Wang, Holiverse Yang, and Zhi-Hao Zhang. LeanCat: A benchmark suite for formal category theory in lean (part I: 1-categories). arXiv:2512.24796, version 2, 2026.
- [12] Lean Community. Theorem proving in lean 4: Propositions and proofs. https://lean-lang.org/theorem_proving_in_lean4/Propositions-and-Proofs/, 2026. Online documentation, accessed September 13, 2026.
- [13] Pawan Sasanka Ammanamanchi, Siddharth Bhat, and Stella Biderman. Faults in our formal benchmarking: Dataset defects and evaluation failures in Lean theorem proving. arXiv:2606.29493, 2026.
- [14] Xinge Liu, Terry Jingchen Zhang, Bernhard Schölkopf, Zhijing Jin, and Kristen Menou. Stargazer: A scalable model-fitting benchmark environment for AI agents under astrophysical constraints. arXiv:2604.15664v2, 2026.
- [15] Shivam Rawat and Lucie Flek. Plausible but wrong: A case study on agentic failures in astrophysical workflows. arXiv:2604.25345, 2026.
- [16] Andrew Cumming. Detectability of extrasolar planets in radial velocity surveys. *Monthly Notices of the Royal Astronomical Society*, 354:1165–1176, 2004.
- [17] Mathias Zechmeister and Martin Kürster. The generalised Lomb–Scargle periodogram: A new formalism for the floating-mean and keplerian periodograms. *Astronomy & Astrophysics*, 496:577–584, 2009.
- [18] Lean Community. The lean language reference: Validating a lean proof. <https://lean-lang.org/doc/reference/latest/ValidatingProofs/>, 2026. Online documentation, accessed September 21, 2026.

A Checking the mathematics results

Review records. The historical checklist has 525 records, including 92 quota failures without a trace. Re-review selects 213 previously labelled records and 121 abstentions. These passes retain 309 primary labels; excluding 11 labels based on `native_decide` alone leaves 298. The model-specific fractions in Section 6 use this retained set. Each reviewed trace record is a configuration-specific task trial, not an independent theorem. The checklist also includes historical configurations outside the 608-run experiment inventory. Codex supplied the initial labels. An extractor and sceptical reviewers revisited them. Blind agreement and later reconciliation use the historical 213 records, before those exclusions. Later reviewers saw earlier judgments. Exact reviewer model versions, complete prompts and frozen input bundles are not preserved. Human review covered representative and disputed cases; annotation was not exhaustive. The retained fractions referenced in Section 6 are: *Tool Feedback Neglect* in 25/53 Codestral and 18/48 Devstral records; *Domain Knowledge Deficit* in 16/29 GPT-5.4 records from the focused Critic comparison and 16/30 GPT-5.4-mini records; and, in the first astrophysics batch, 22 *Reasoning Failure* and ten *Tool Feedback Neglect* assignments among 45 reviewed failed runs. These are fractions of selected labelled records, not rates over all runs.

A harmful review. The three matched rewrite cases are trials 0, 6 and 9 of `easy_fatem_115`. In both configurations, the Critic receives an accepted proof with the same deprecation warning. The Codestral Critic replaces `Transitive` with `IsTrans`, but the new code does not type-check. In trial 9, later submissions repeat rejected code after an explicit type error. The run ends stuck. The matched GPT-5.4 runs approve the unchanged proof. The initial rewrite and later ignored feedback are separate events; repeated repair is not by itself a State Tracking Failure.

Kernel validation. Each accepted proof is re-compiled against Mathlib v4.30.0, checked to be sorry-free, checked for the task’s declared statement, and passed through `#print axioms`. Of the 300 approved runs in Table 4, 287 pass and 13 fail: twelve are `native_decide` axiom escapes on `medium_fateh_010`, and one (`easy_fatem_115`, trial 2, Codestral Critic) is axiom-clean but proves a different statement. The twelve escaped axioms assert true arithmetic, and `decide` proves the same goals axiom-free, so the issue is what the verdict claimed rather than a false proof [13, 18]. The earlier review’s 79 replay failures came from narrower imports in the offline harness; under the agents’ own environment only these 13 fail.

Matched Critic outcomes. Among the 214 paired runs, both teams reach approval in 138 pairs, only the GPT-5.4-Critic team in 18, only the Codestral-Critic team in six, and neither in 52. Both produce an accepted check in 153 pairs. These pairs share tasks and trial indices, not identical generated proofs. They cover only 20 statements; seven of the 15 missing approvals in the Codestral-Critic team concern one statement. The three damaging rewrites above are also trials of one statement.

B Experimental settings and dataset limits

B.1 Mathematics

Five-configuration experiment. Each configuration uses the same 20 statements, with three runs per statement. This gives 60 runs per configuration.

Focused Critic experiment. The Reasoner and Engineer use GPT-5.4. The Critic uses either GPT-5.4 or Codestral. Both teams use the same 16 statements: fifteen have ten runs each, and one has four. This gives 154 runs per team.

We pair runs by statement and trial index within each experiment. The prompts, temperature of 0.2 and 30-turn limit are shared. Each run generates its own solution and proof, so paired runs do not give the two Critics identical inputs.

B.2 Astrophysics

Tasks and runs. Table 5 gives the task coverage. Each experiment uses the listed model in all three roles.

Model	Easy	Medium	Hard	Runs
GPT-5.4	20	29	0	145
GPT-5.4-mini	0	8	8	16
GPT-4o-mini	20	28	8	56
GPT-4o	0	7	5	12

Table 5: Distinct tasks at each difficulty level and total runs. GPT-5.4 has three runs on 48 tasks and one on the remaining task. The other models have one run per task. Four incomplete GPT-4o logs are excluded.

Grading and filtering. The GPT-5.4 batch uses a matching threshold of 0.80. The other three experiments use 0.512. They retain tasks whose reference fit passes this matching threshold. This filter does not check the residual-error condition. GPT-5.4-mini uses tasks with at least two planets; GPT-4o-mini also includes one-planet tasks.

Run limits. The limits depend on task difficulty (Table 6). Three GPT-4o hard-task runs instead use a 60-turn limit.

Difficulty	Maximum turns	Maximum submissions
Easy	30	3
Medium	50	5
Hard	90	10

Table 6: Run limits in our astrophysics experiments.

Tools. Stronger models, including Claude Fable 5 and GPT-6 Astra, attempted tasks and helped develop tools during preparation. During evaluation, the agents use fixed tools. They cannot write Python or change the fitter. This restriction differs from STARGAZER’s original coding environment [14, Section 3.2].

Task coverage, filtering, grading thresholds and run limits differ between these experiments. Their overall results cannot isolate the effect of the model.

B.3 Why we checked the dataset and grader

Known generating parameters do not show that the required system can be recovered from the observations. We therefore tested the fitter with true periods, submitted the generating parameters directly, checked reference fits against every grading condition, and regraded fitted systems that differed from the hidden truth.

Appendix C reports these diagnostic experiments, their results and the conclusions they support. They provide the evidence for our dataset criticism and help separate task and tool limits from later agent mistakes.

C Astrophysics dataset checks

We checked the data, fitter and grader before assigning failures to the agents. During preparation, stronger models attempted the tasks and helped develop the tools. We also ran offline numerical checks on the datasets behind the failed runs. These checks used hidden parameters for diagnosis; the evaluated agents did not see them.

C.1 Can the provided fitter produce a passing answer?

We tested the 17 datasets behind the 45 reviewed failures in batch 1. For each dataset, we gave `rv_fit` the true orbital periods as starting inputs. The fitter then adjusted the orbital parameters against the original noisy observations. We graded the returned fit using all four conditions: BIC improvement, residual error, planet count and parameter matching.

We then submitted the generating parameters directly, without fitting. This separates failure of the fitting procedure from rejection of the answer used to generate the data.

Diagnostic check	Datasets passing
Fit the observations, starting from the true periods	0/17
Submit the generating parameters directly	16/17

Table 7: Diagnostic results under the batch-1 grader. The unit is a dataset, not an agent run.

Sixteen reference fits fail parameter matching. The remaining fit, on seed20, fails the residual-error condition. The generating parameters also fail that condition on seed20.

These results show that correct starting periods do not ensure that the tested fitter passes. They support a limitation of our fitter-grader combination. They do not prove that every possible fit must fail.

C.2 Why does fitting move away from the generating parameters?

We examined seed12 more closely. The fitted period is 5.173 days, close to the generating period of 5.164 days. Yet the match score is 0.383, below the required 0.80.

We separated the match distance into its period, amplitude, eccentricity, phase and curve terms. The period difference contributes little; the other orbital properties explain most of the rejection. Starting the optimiser at the generating parameters still returns the rejected noisy-data fit. In a noise-free control, the fitter recovers the injected orbit.

This shows why a successful numerical fit can still fail parameter recovery. Limited observations and noise affect which orbital properties can be determined. Radial-velocity detectability depends on signal strength, orbital shape and observation duration [16]. Our control supports this explanation for the tested case; it does not establish a global recovery limit.

C.3 Does the task filter check the full grader?

For the filtered GPT-5.4-mini batch, we checked the stored reference fits against the residual-error condition as well as matching. Three reference fits pass the matching filter but fail the error limit.

Task	Reference RMS (m/s)	Allowed RMS (m/s)
seed13	0.857	0.847
seed89	1.694	1.490
seed500008	1.169	0.979

Table 8: Reference solutions retained by the matching filter but rejected by the residual-error condition.

The limit is 1.5 times the median reported measurement error. The generated observations also contain noise that these reported errors do not fully include. Checking matching alone therefore misses a separate grading problem.

For five other tasks, we compared the observation span with the true periods and inspected signal amplitudes relative to noise. They contain weak or partly observed planets. Passing reference fits exist, but start from the true periods. Their success does not show that an agent can find those periods from the observations alone.

C.4 Does a passing grade mean physical recovery?

We also regraded fits already produced in the seed6 and seed27 trajectories. Some pass with a short-period component replacing the true long-period planet.

The matching rule can omit poorly matched planet pairs from its average, even when the submitted planet count is correct. STARGAZER discusses this limitation and proposes a stricter normalisation [14, Appendix B.1]. Our rescoring shows its practical consequence: a passing answer can still describe the wrong planetary system.

C.5 What these checks support

Known generating parameters give an answer key. Our experiments show why that alone is insufficient for evaluating agent failure: the reference fitter can fail, the task filter can miss a grading condition, and a physically different system can pass.

STARGAZER reports task filtering and numerical baselines [14]. Our findings concern the local datasets and tools used here. They motivate checking every grading condition and reporting a reference recovery procedure that uses only information available to the agent.

Our fixed-tool restriction is a separate system design issue. STARGAZER’s original environment allows agents to write and execute code [14, Section 3.2]. A failure under our restricted workflow cannot by itself establish failure under that environment.

D Astrophysics diagnostic checks

What the local fitter searches. The velocity model sums Keplerian planet signals and one constant offset per instrument. The fixed fitter uses multistart weighted least squares and restricts each period to within 20% of the supplied guess. The periodogram supplies candidate periods; it does not determine all orbital parameters [17]. A true-period reference fit therefore tests this fitting procedure with privileged initial information. It is not a proof of the best possible fit or a blind solver available to the agent.

The dataset and reference-fit checks are reported in Appendix C.

The five weak or partly observed systems are seed9, seed122, seed51, seed27 and seed37. Passing reference fits exist, but use the true periods. They do not establish that the agents can infer every required planet from the observations alone.

The three team cases. On seed45, the Reasoner interprets a residual check on an unfitted guess as a failed fit. The Engineer then obtains a fit with RMS 2.527 m/s, below the 3.770 limit, but rejects it using the 19.340 m/s residual from unfitted inputs. The mistaken diagnosis persists and no submission follows. On seed21, the Reasoner treats generic phase/epoch advice as a diagnosis and the team abandons useful alternatives. On seed500060, the Engineer omits the addressed handoff after a four-planet fit that passes offline checks. This is a missed handoff without a separately established secondary error.

Matching and hidden-truth rescoring. The local matching rule discards assigned planet pairs with distance $d > 5$. When planet counts agree, it averages $\exp(-d)$ over the remaining pairs. An unmatched true planet contributes neither a score nor a denominator term. A count difference also incurs a score penalty and fails the separate count check. STARGAZER describes this limitation and proposes normalising by the true planet count [14, Appendix B.1]. A passing grade therefore need not establish physical recovery of every planet. Our offline subset analysis selects only from planets already returned by the fitter and applies all grading conditions. It makes no new fits, and uses information unavailable to the agent. Its loss counts measure available fitting output versus submitted answers, not the number of Critic errors.